



RUHR-UNIVERSITÄT BOCHUM

PIPELINING

Computer Architecture

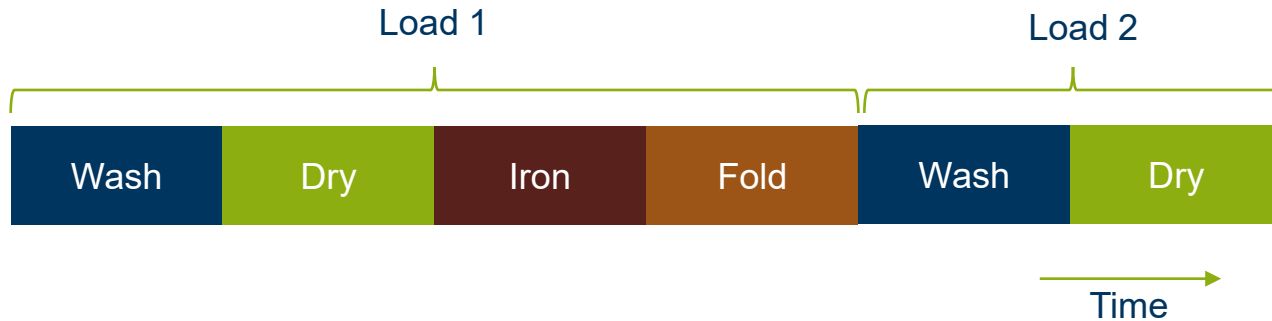
Agenda

- What is pipelining?
- The MIPS 5-stage pipeline
- Pipeline hazards
 - Data hazards
 - Control Hazards
- Scoreboarding

Pipelining Example

Alex and Bob come home after travelling and have lots of clothes to wash

They only have 6 hours before meeting up with friends, how many loads of laundry can they do today when each step takes an hour?



Only one and a half?

Pipelining Example



Three!

Each step is independent from the others and all can thus be carried out simultaneously

Why Do We Pipeline in Computer Systems?

Idle components decrease performance

The instruction throughput should be as high as possible

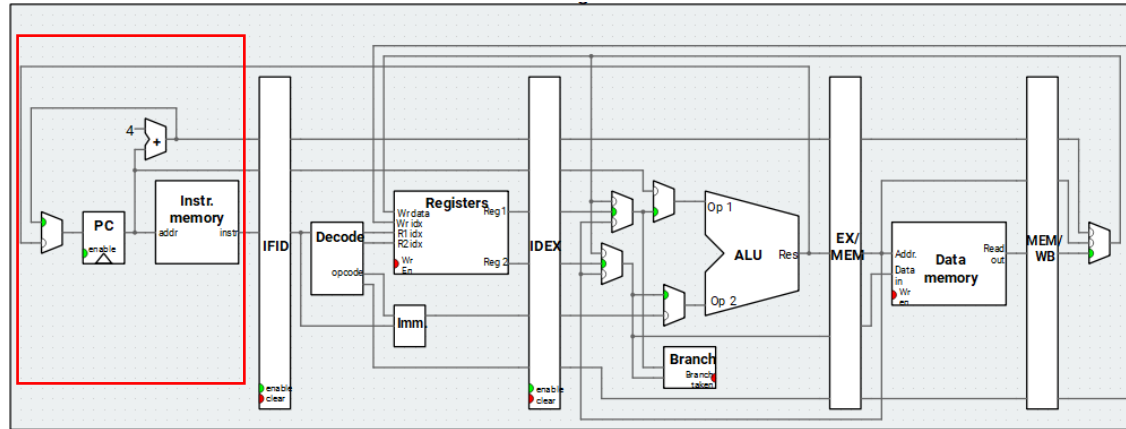
Why Pipelining?

- Each instruction is divided into multiple stages
- Possible since all instructions have the same length

MIPS 5-stage pipeline

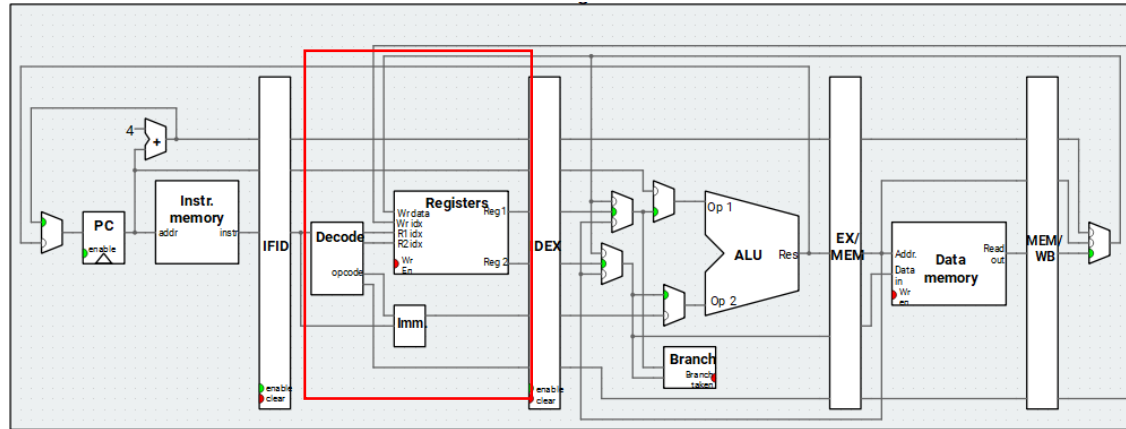
Instruction Fetch (IF)

Load instructions from RAM to instruction memory
Increase program counter



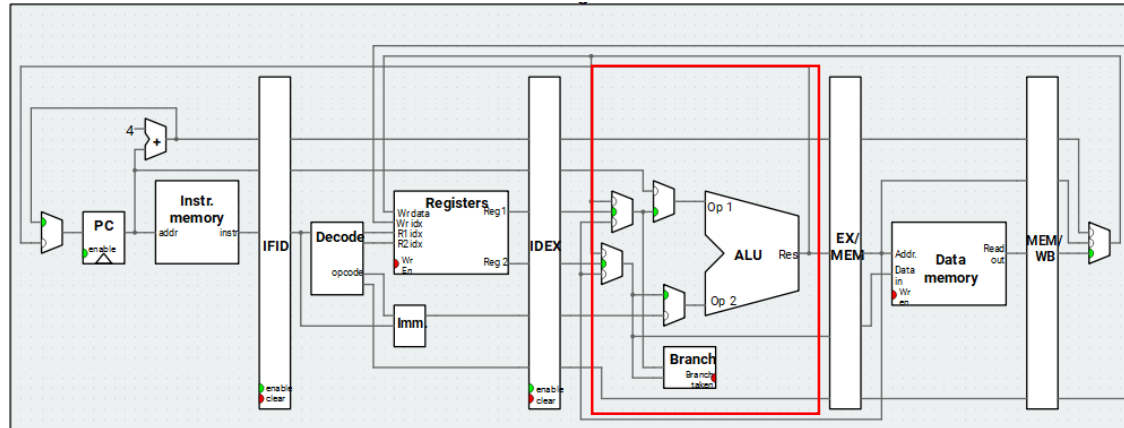
Instruction Decode (ID)

Decode instructions into control signals that can be used by the ALU
Fetch all necessary operands



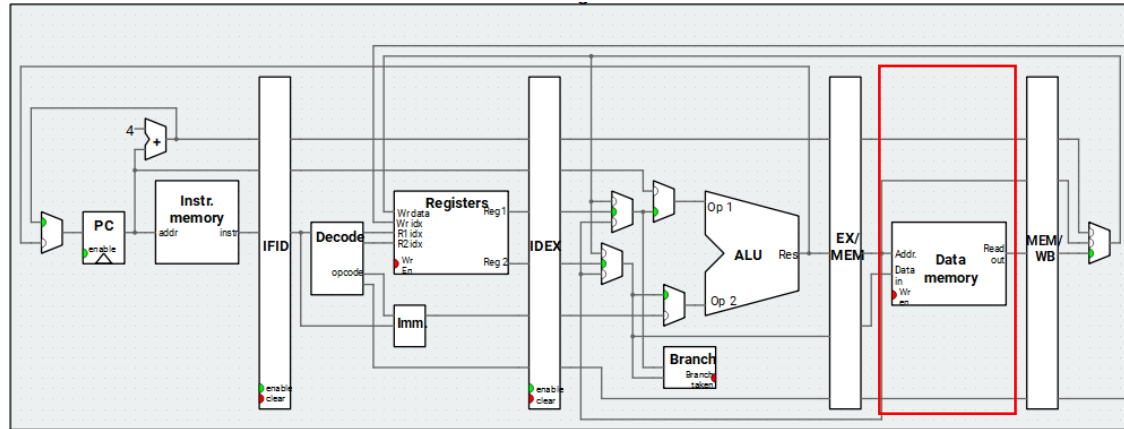
Execute (EX)

Compute results



Memory Access (MA)

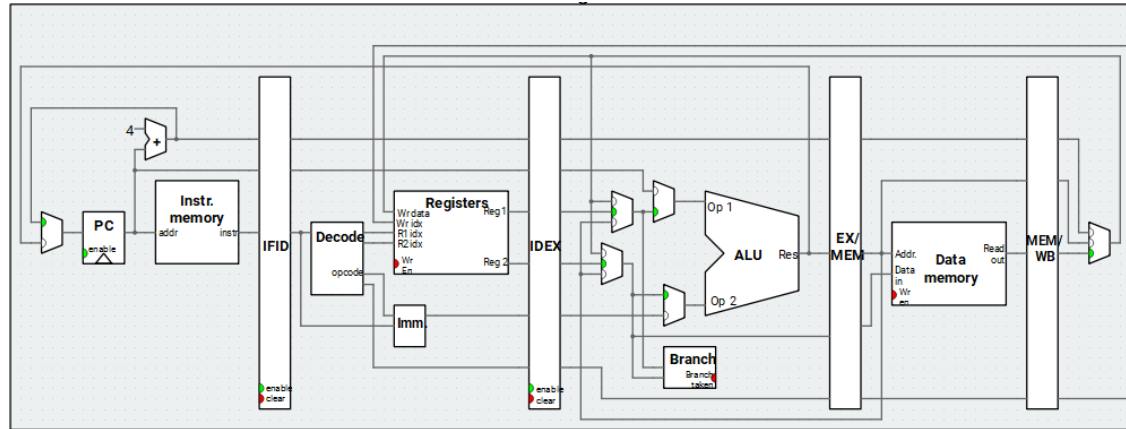
Access and alter memory, if necessary



Write back (WB)

Write result into specified register

Only after the WB-Phase is completed can the result be accessed



Speedup

t : Time to complete an instruction without pipelining

n : Number of instructions to be completed

k : number of pipeline stages

Formula:

For $k = 1$: $n \cdot t$

For $k > 1$: $t + \frac{(n-1)t}{k}$

For $m \gg k$: Speedup approaches k

Limitations

Instructions can depend on result of other instruction that is computed to late when pipelining

➡ Hazards

Some instructions are more complex than others and take more time to complete, particularly in CISC

➡ Pipelining is harder to implement

Pipeline Hazards

Data Hazards

Instructions can depend on data from other instructions

I1: ADDI t0, t0, 49

I2: XOR t1, t0, t1

I3: SLLI t3, t3, 2

*I*₂ needs the
result from *I*₁
here but it
hasn't been
computed yet

IF	ID	EX	MEM	WB
<i>I</i> ₁				
<i>I</i> ₂	<i>I</i> ₁			
<i>I</i> ₃	<i>I</i> ₂	<i>I</i> ₁		
	<i>I</i> ₃	<i>I</i> ₂	<i>I</i> ₁	

Solution: NOPs

IF	ID	EX	MA	WB
I_1				
NOP	I_1			
NOP	NOP	I_1		
NOP	NOP	NOP	I_1	
I_2	NOP	NOP	NOP	I_1
I_3	I_2	NOP	NOP	NOP
	I_3	I_2	NOP	NOP



Now I_2 reads the correct data from the register

Solution: NOPs

IF	ID	EX	MA	WB
I_1				
I_2	I_1			
I_3	I_2	I_1		
	NOP		I_1	
	NOP			I_1
	NOP			
	I_3	I_2		

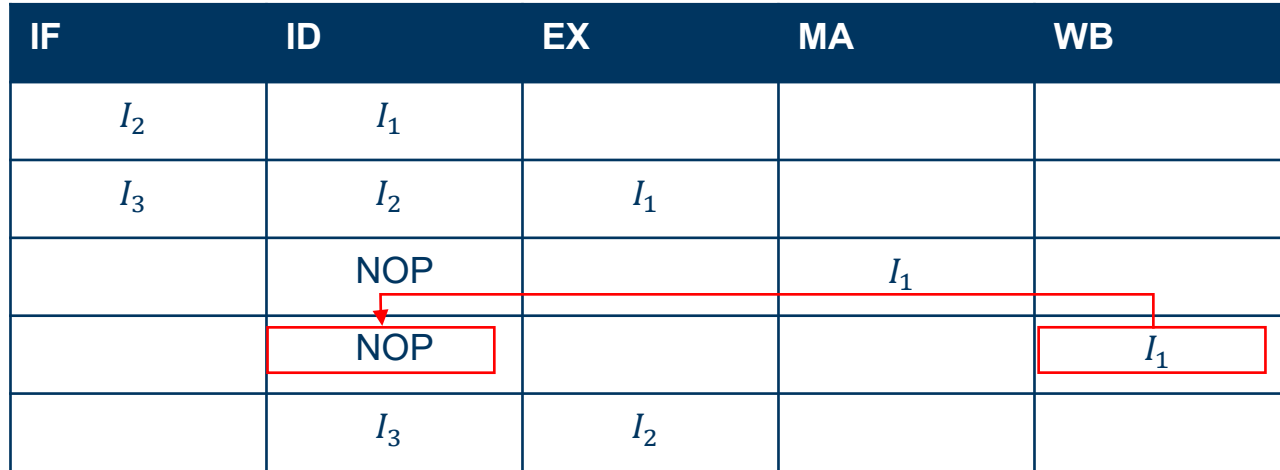


Now I_2 reads the correct
data from the register

Solution: Forwarding

Add circuit that passes result directly from the WB-Phase to the ID-Phase

IF	ID	EX	MA	WB
I_2	I_1			
I_3	I_2	I_1		
	NOP		I_1	
	NOP			I_1
	I_3	I_2		



Solution: Rearranging

Rearrange the instructions so that later instructions are carried out instead of NOPs

IF	ID	EX	MA	WB
I_3	I_1			
NOP	I_3	I_1		
I_2	NOP	I_3	I_1	
	I_2	NOP	I_3	I_1
		I_2	NOP	I_3

Types of Dependencies

True Dependency: Read after Write (RAW)

Example:

I1: ADDI *t0*, *t1*, 38

I2: OR *t2*, *t0*, *t2*

The result of *I1* is written into register *t0* which is then accessed by *I2*

 *I2* depends on the value of the operation from *I1*

Types of Dependencies

Anti Dependence: Write after Read (WAR)

Example:

I1: XOR *t0*, *t1*, *t2*

I2: ADDI *t1*, *t3*, 4

I1 accesses the data from register *t1* which is then updated by *I2*

➡ *I1* depends on the value of *t1* before the write of *I2*

Types of Dependencies

Output Dependence: Write after Write (WAW)

Example:

I1: SLLI *t0*, *t1*, 4

I2: AND *t0*, *t2*, *t3*

The result of *I1* is written into *t0* and afterwards the result of *I2* is also written into *t0*



The writes have to happen coordinated with all other instructions that depend on the different values of *t0*

Control Hazards

Which instruction is executed next depends on branch instructions

Example:

I1: beq t0, t1, equal

I2: addi t0, t1, 4

equal:

I3: addi t0, t1, 7

Decide if it is more likely that the branch is taken or not and load the next instruction accordingly

If the prediction was not correct:
Flush pipeline and load other instructions

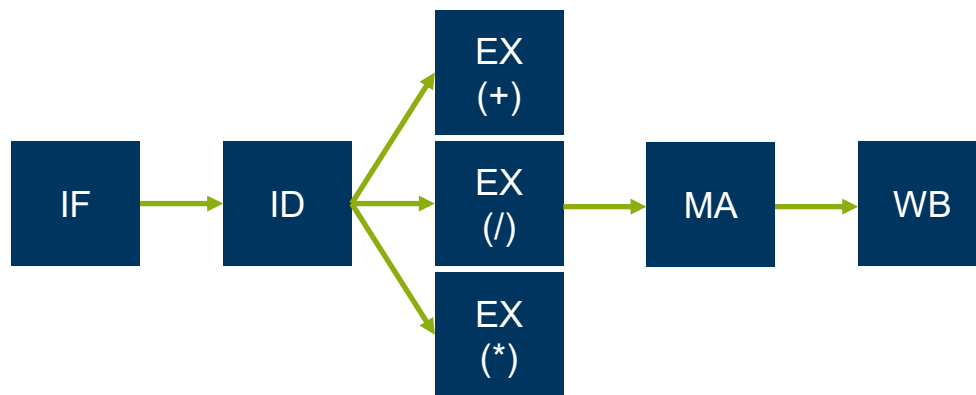
Scoreboarding

Multiple Functional Units

So far, we only used one unit to execute instructions

Now:

- Use multiple functional units
- Let Instructions execute in parallel



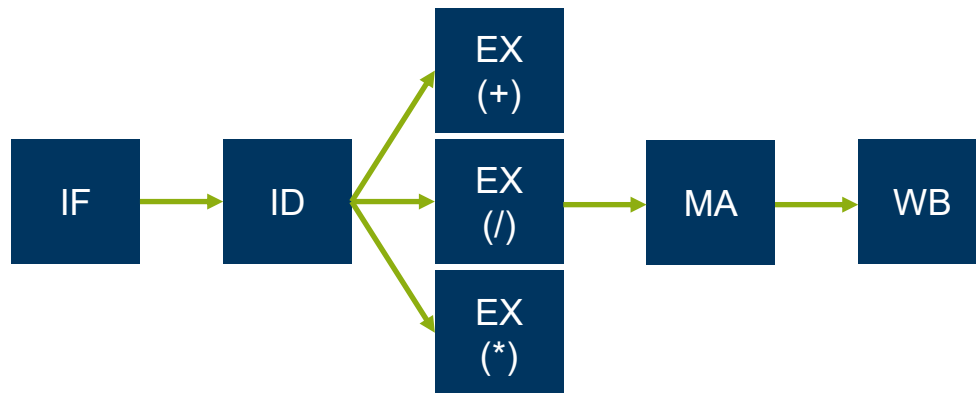
Does this work?

Does this Work?

I1: ADD t0, t2, t1

I2: MUL t4, t0, t2

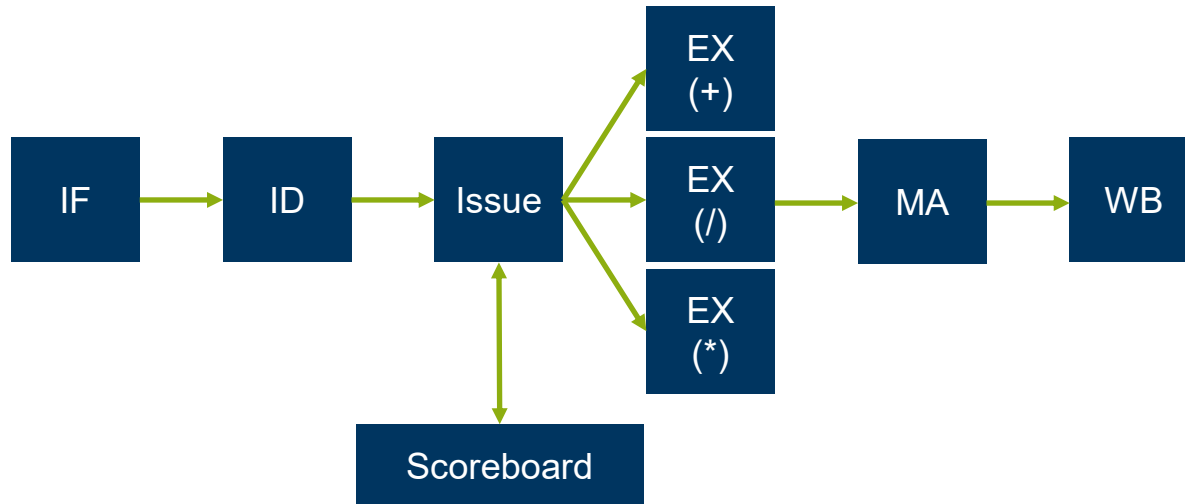
Register values are fetched in the ID-Phase, *I2* would run on an outdated value of t0 (true dependency)



➡ Not yet

The Issue-Stage

Add another stage that makes use of a scoreboard



The Scoreboard

Table that keeps track of which FU's are in use



Functional Unit	Occupant
EX(+)	I1
EX(/)	---
EX(*)	---

Table that keeps track of which registers are reserved for writing



Register	Reserved for
t0	I1
t1	---
t2	---
t3	---
t4	---

Scoreboard after issuing I1

The Scoreboard

Issuing I2:

MUL t4, t0, t2



FU	Op1	Op2	Op3
free	free	I1	free

Execution of I2 has to wait until all operands are free and the required FU is available

Register	Reserved for
t0	I1
t1	---
t2	---
t3	---
t4	---

Functional Unit	Occupant
EX(+)	I1
EX(/)	---
EX(*)	---

The Scoreboard

When an instruction is completed the blocked resources are freed in the scoreboard

Instructions that waited for these can now be executed

While one instruction is waiting others might be ready to go and are executed immediately

Scoreboarding – WAR hazards

Out-of-Order execution introduces risk of WAR hazards

I_1 : MULI t1, t1, 7

I_2 : ADDI t0, t1, 3

I_3 : ADDI t1, t3, 5

Instruction I_2 must wait for I_1

Instruction I_3 can execute immediately. If it completes before I_1 , I_2 will read a wrong value

Solution: stall write-back until all older instructions that read from the output register has read their inputs

Summary

- Pipelining increases efficiency by reducing the time each component of the CPU rests idle
- When Pipelining it is important to keep track of hazards
 - Data hazards: Dependencies
 - Control hazards: Branches
- To further increase the instruction throughput multiple functional units can be used, but a measure of consistency is required -> scoreboarding